

Smart City Audio

Un microfono che ascolta la città senza registrarla

Comune di Trento, 2024 — il Garante privacy sanziona un progetto di sorveglianza acustica: i microfoni trasmettevano audio in chiaro, riconoscibile.

Il suono non esce mai dal dispositivo

Approccio tradizionale

Il microfono trasmette l'audio a un server, che lo analizza.
Voci e conversazioni viaggiano in chiaro: sono dati personali.

Approccio adottato

Il microcontrollore classifica a bordo e trasmette solo un'etichetta.
Nessun audio lascia il dispositivo durante il funzionamento normale.

Esce dal dispositivo:

```
{"t1":"Vetri","p1":0.71,"stato":"POSSIBILE_VANDALISMO"}
```

Una stringa di quaranta byte. Non è ricostruibile in audio, non contiene voce.

Un core, 264 KB, nessuna virgola mobile

133 MHz

Cortex-M0+
senza FPU

264 KB

RAM totale
220 usati

2 MB

flash
modello e codice

Il vincolo che decide tutto

Il Cortex-M0+ non ha unità in virgola mobile: ogni moltiplicazione float è emulata via software e costa decine di cicli. Il progetto è stato portato a 200 MHz.

FreeRTOS, priorità fisse + preemption

Perché un RTOS

Tre attività con periodi diversi — 32 ms, 496 ms, sporadica — che devono convivere su un core, con prelazione immediata della più urgente.

Perché FreeRTOS

Porting ufficiale per RP2040, integrazione con SDK e DMA, footprint contenuto: 80 KB di heap assegnati (utilizzati <50).

Scheduler: priorità fisse con prelazione

L'assegnazione segue Rate Monotonic — periodo più breve, priorità più alta. Non è una convenzione: coincide con l'ordine di urgenza delle scadenze, perché il task con il periodo più corto è anche quello con la scadenza dura.

Dalla pressione sonora all'etichetta



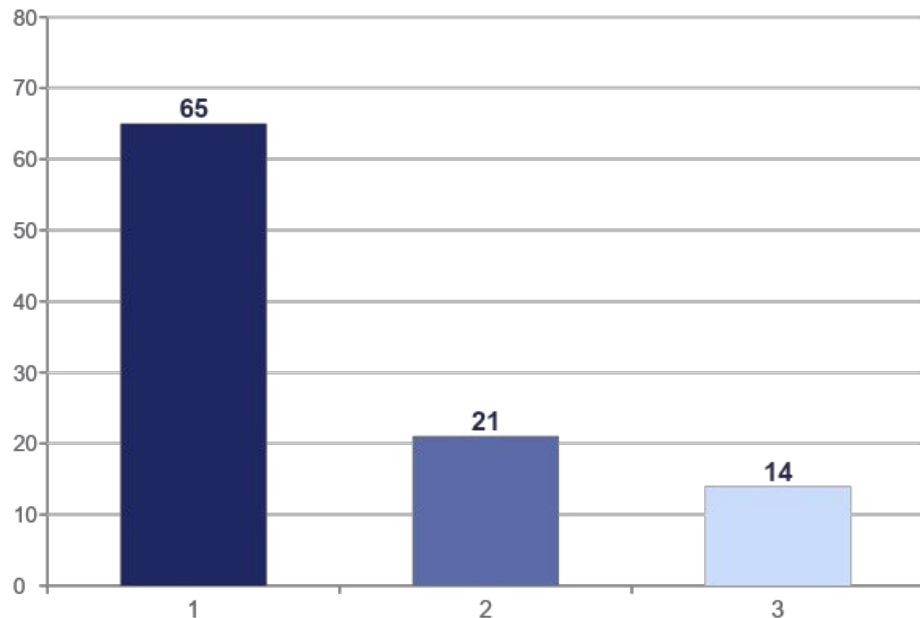
Ogni blocco ha un costo misurato e una scadenza. È questo che rende il progetto un problema di tempo reale e non solo di riconoscimento audio.

Tre task, tre priorità - RM

Task	Priorità	Attivazione	Contenuto
Task_DSP	3	periodico, 32 ms	Acquisizione, FIR, decimazione, FFT, mel
Task_ML	2	periodico, 496 ms	Inferenza CNN e macchina a stati
Task_Serial	1	aperiodico (Background Service)	Trasmissione seriale

Più la ISR del DMA, a priorità hardware bassa perché chiama API del kernel, e i task Idle e Timer di FreeRTOS.

Dove va il tempo



Task_DSP

Response Time = $C = 20,8$ ms

$T = 32$ ms

→ $U = C/T = 65\%$

Task_ML

Response Time = 298ms, $C = 105$ ms

$T = 496$ ms

→ $U = C/T = 21\%$

Il criterio di Rate Monotonic non basta

Limite di Liu e Layland

$$U \leq n (2^{(1/n)} - 1)$$

$$\text{con } n = 2 \rightarrow U \leq 0,828$$

Il sistema è a 0,86: la condizione NON è soddisfatta.

Ma è solo sufficiente

Il criterio garantisce la schedulabilità se soddisfatto, non la esclude se violato. Superarlo obbliga all'analisi esatta dei tempi di risposta, non a riprogettare.

Il caso peggiore va calcolato.

RTA

$$R = C + \sum \lceil R / T_j \rceil \cdot C_j \quad \text{con } j \text{ sui task a priorità maggiore}$$

Task_ML: $R = 105 + \lceil R / 32 \rceil \cdot 20,8$

$$R_0 = 105$$

$$R_1 = 188,2$$

$$R_2 = 229,8$$

$$R_3 = 271,4$$

$$R_4 = 292,2$$

$$R_5 = 313,0$$

$$R_6 = 313,0 \leftarrow \text{punto fisso}$$

313 ms

tempo di risposta nel caso peggiore

contro una scadenza di 496 ms

→ sistema schedulabile

Sotto sovraccarico si scarta, non si accumula

Il sistema salva una registrazione di 2 secondi quando riconosce un evento pericoloso.

Il problema

Una clip occupa il canale seriale per secondi. Eventi ravvicinati la richiederebbero più volte.

La scelta

Tempo morto di 10 s e buffer congelato.
Le richieste in eccesso vengono rifiutate, non messe in coda.

Il motivo

Accodare significherebbe riversare, fra un minuto, un evento vecchio di un minuto.

Nessuna politica di scheduling risolve un sovraccarico: non c'è modo di eseguire più lavoro di quanto ne stia nel tempo disponibile.

La rete neurale, in breve

31 KB

modello
quantizzato int8

107 K

moltiplicazioni
per inferenza

88,9%

accuratezza
sul dataset

8

classi
acustiche

Architettura

Tre convoluzioni standard, pooling, uno strato denso.

Il divario che conta

88,9% sul dataset, 42,7% sulle registrazioni fatte con il microfono reale. L'adattamento di dominio (fine tuning) lo ha riportato al 65,2%.

Dove la rete non può arrivare

Alcuni eventi non esistono dentro una finestra da un secondo.

Sparo o fuochi?

Acusticamente identici. Li distingue solo la ripetizione nel tempo.

Incidente

Non è un suono: è una sequenza — stridio, impatto, vetri.

Un vincolo categorico si scrive, non si fa dedurre

Livelli smorzati per le classi continue, eventi con tempo morto per quelle impulsive, isteresi di 5 s sugli stati. Otto stati, dal fondo urbano al possibile incidente. Costo: decine di microsecondi per ciclo.

Un pulsante, nessun task in più

Premendolo, il PC riproduce un campione: serve a mostrare il riconoscimento dal vivo.

Letto da Task_DSP

Periodo fisso di 32 ms e mai bloccante: il campionamento è deterministico.

Antirimbalzo gratuito

Tre letture consecutive a 32 ms, in entrambi i versi. Nessun timer: la base dei tempi è la periodicità del task.

Costo trascurabile

Due letture di un registro GPIO. Decine di nanosecondi su un budget di 32 ms: l'analisi non cambia.

Bilancio Finale

Raggiunto

- Schedulabile, dimostrato analiticamente.
- In ore di funzionamento: 0 overrun ADC, 0 finestre scartate.
- L'audio non lascia mai il dispositivo.

Limiti

- L'accuratezza reale è 65%, contro l'89% sul dataset: dataset di training non dei migliori
- Classi come ambiente urbano e veicoli restano poco separabili: il microfono su ADC a 12 bit è il collo di bottiglia.